

INTRODUCTION AU LANGAGE



Moussa LO
M2ITIC/IFOAD 2014/2015
Université Gaston Berger de Saint-Louis

Bibliographie Java (1/2)

- **Le langage Java**, K.Arnlod, J. Gosling, Thomson, 1996.
- **Java La synthèse – Des concepts objet aux architectures Web**, G. Clavel, et al., Dunod, 2000.
- **Introduction à la programmation objet en Java**, J. Brondeau, Dunod, 1999.
- **Le langage Java – Programmer par l'exemple**, T. Leduc, D. Leduc, Technip, 2000.
- **Algorithmique et programmation objet en Java**, V. Granet, Dunod, 2001.

Bibliographie Java (2/2)

- **Initiation à l'algorithmique objet**, A. Cardon, C. Dabancourt, Eyrolles, 2001.
- **Thinking in Java**, B. Eckel, Prentice Hall, 2000.
Disponible sous forme électronique à l'URL
<http://www.BruceEckel.com>
- **Le poly de Java**, H. Garetta, Polycopié, Université de la Méditerranée, 2003.
- Site Java de Sun : <http://java.sun.com>



RAPPELS DE POO

Rappels de POO

- Généralement, les données sont plus stables et pérennes que les traitements qu'on leur applique.
- Donc, il est préférable de structurer un système par rapport à ses données plutôt que par rapport à ses fonctions.
- ➔ Concept de POO : tout est dirigé par les données.
- Les concepts principaux de la POO :
 - ➔ Encapsulation
 - ➔ Héritage
 - ➔ Polymorphisme

Rappels de POO : Encapsulation

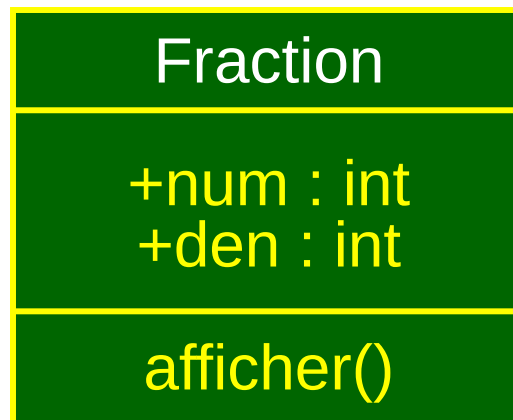
- Mécanisme consistant à empaqueter données et traitements au sein d'une même structure en ne la rendant accessible que par le biais des opérations (**interface**) laissées visibles à cet usage.
- On empêche l'accès aux données par un autre moyen que les services proposés.
- Cette structure qui encapsule les données et les traitements s'appelle l'**objet**.
- L'encapsulation permet donc de garantir l'intégrité des données contenues dans l'objet.

Rappels de POO : Notion d'objet

- Un objet se définit par un triplet :
 - un **identificateur** (*OID* ou *Object Identifier*) qui permet de le référencer de façon unique.
 - des données (**attributs**) caractérisant l'objet. Ce sont des variables stockant des informations d'état de l'objet.
 - et des procédures (**méthodes**) agissant sur ces données. Elles caractérisent son comportement, c-à-d l'ensemble des *opérations* que l'objet est à même de réaliser. Ces opérations permettent de faire réagir l'objet aux sollicitations extérieures (ou d'agir sur les autres objets).

Rappels de POO : Notion de classe

- Une *classe* est la description d'une famille d'objets ayant même structure et même comportement.
- Elle regroupe un ensemble d'*attributs* (données représentant l'état de l'objet) et un ensemble de *méthodes* (opérations applicables aux objets).
- Objet = instantiation d'une classe.



Rappels de POO : Notion de message et de liaison

- L'activation de méthode se fait par envoi d'un **message** à l'objet concerné (objet récepteur).
- Message = récepteur + signature de méthode
- Signature de méthode = (nom méthode + paramètres)
- **Liaison** : mécanisme consistant à sélectionner le code de la méthode à activer lors de la réception d'un message par un objet récepteur :
 - Liaison **statique** : réalisée à la compilation du code source
 - Liaison **dyamique** : réalisée à l'exécution (*cas de Java*)

Rappels de POO : Notion d'héritage

- Mécanisme de transmission des propriétés d'une classe à une autre (sous-classe) par dérivation.
- *Spécialisation* : L'héritage permet de propager des spécifications d'une classe "générale" vers des sous-classes "particulières" en offrant la possibilité de les enrichir successivement.

Rappels de POO : Redéfinition de méthode

- Réécrire l'implémentation d'une méthode héritée sans modifier sa signature (seul l'objet récepteur diffère)
- Une méthode héritée peut être **redéfinie** dans la sous-classe: on conserve la même signature et le même type de retour.
- On peut étendre le comportement d'une méthode héritée: la méthode de la super-classe reste accessible pour les objets de la sous-classe grâce au mot-clé **super..**

Rappels de POO : Surcharge de méthodes

- Redéfinition de la signature et du code d'une méthode héritée
- La **surcharge** concerne 2 méthodes de même nom mais avec des signatures différentes:
 - dans la définition d'une même classe (exple: constructeurs)
 - dans la surcharge des méthodes d'une super-classe.

Rappels de POO : Notion de polymorphisme

- Faculté d'une méthode donnée à pouvoir être appliquée à divers objets instanciant chacun des classes différentes.
- C'est le cas lorsqu'une sous-classe hérite d'une méthode sans la surcharger.
- La liaison dynamique consiste dans ce cas, lors de l'exécution, à parcourir de façon ascendante le graphe d'héritage pour trouver l'objet récepteur.

La première méthode dont la signature correspond à celle du message sera exécutée.



CLASSES ET OBJETS EN JAVA

Définition d'une classe

public class NomDeLaClasse {

- *déclarations des champs (attributs, propriétés)*
- *déclarations des méthodes*

}

public class Fraction {

// champs

public int num ;

public int den ;

// méthodes

public void afficher () {

if (this.num % this.den == 0)

System.out.println(this.num/this.den

) ;

else

System.out.println(this.num+"/"+this.den) ;

}

Le désignateur (ou auto-référence) this

- **this** est une pseudo-variable qui permet :
 - ✓ de faire référence à l'objet courant (celui qu'on est en train de définir).
 - ✓ ou de désigner ses attributs ou ses méthodes.
- **this** est généralement utilisé pour lever l'ambiguïté sur un identificateur lorsque le paramètre d'une méthode porte le nom de l'attribut de l'objet qu'il est chargé de modifier.
- *Exemple :*

```
void fixerNumerateur (int num) {  
    this.num = num ;  
}
```

Utilisation d'une classe

```
public class FractionUtil {  
    public static void main (String[] args) {  
        Fraction f = new Fraction();  
        f.num = 8; f.den = 5;  
        f.afficher();  
    }  
}
```

- Instanciation avec l'opérateur *new*
- Java s'occupe de la destruction des objets non utilisés (mécanisme de *garbage collector*).
 - pas besoin de “destructeurs”.

Exemple d'application Java avec le JDK

- Le fichier source prend le nom de la classe qu'il définit avec l'extension **.java**

==> FractionUtil.java.

- La compilation génère un fichier binaire Java avec l'extension **.class**.

javac FractionUtil.java

==> FractionUtil.class

- Ce fichier peut être exécuté grâce à un interpréteur java.

java FractionUtil

Constructeurs spécifiques

```
public class Fraction {  
    public int num, den ;  
  
    public Fraction (int n, int d) { this.num=n;  
this.den=d;}  
    public Fraction (int n) { this.num=n; this.den=1;}  
  
    public Fraction inverser() {  
        if (this.num == 0) return null;  
        return new Fraction(this.den,this.num);  
    }  
}
```

- Les divers constructeurs doivent avoir des signatures différentes (*nombre d'arguments ou types des arguments différents*).

La méthode main(1)

- Premier sous-programme exécuté au lancement d'une application Java.
- Signature:

```
public static void main (String[] args) {  
    ...  
}
```
- Le paramètre est un tableau de chaînes utilisé pour les arguments transmis à partir de la ligne de commande.

La méthode main(2)

Exemple avec arguments:

```
public class somme
{
    public static void main(String[] args)
    {
        int x = Integer.parseInt(args[0]);
        int y = Integer.parseInt(args[1]);
        System.out.println("la somme de "+x+"
et "+y+" est: "+x+y);
    }
}
```

Utilisation:

java somme 12 14

=> la somme de 12 et 14 est: 26

Visibilité des champs et méthodes

- Le principe d'encapsulation permet d'introduire celui de protection des attributs et méthodes.
- **Les Modificateurs de visibilité en Java :**
 - **public** : accessible par toutes les classes. Hérité par les sous classes.
 - **private** : accessible que par les seules méthodes de sa classe. Non hérité.
 - **protected** : accessible par les classes du même *package*. Hérité par les sous classes.
 - **par défaut** : accessible par les classes du même *package*. Hérité par les sous classes que si elles se trouvent dans le même package.

Visibilité : exemple

```
public class Etudiant {  
    public String nom;  
    private String numIns;  
  
    public Etudiant(String nom) {  
        this.nom = nom;  
    }  
  
    // methodes accesseurs  
    public String getNumIns() { return this.numIns; }  
    public void setNumIns(String numIns) {  
        if (numIns==null || numIns.length!=11) return;  
        this.numIns = numIns;  
    }  
  
    public void afficher(){  
        System.out.println("etudiant :"+ nom);  
    }  
}
```

Les variables (1)

- **variables d'instance** : elles déterminent les attributs ou l'état d'un objet donné (*attributs d'instance*)
- **variables de classe** : elles déterminent les attributs ou l'état d'une classe donnée (*attributs de classe*)
- **variables locales** : déclarées et utilisées dans les définitions de méthodes.

Syntaxe:

Type_de_la_variable identificateur_de_la_variable;

Exemples:

- **int** annee;
- **String** nom, prenom, adresse;

Les variables (2)

Class Une_Classe {

- ...
 - *déclarations des variables de classe et d'instance*
 - ...
 - public void une_Methode (déclarations des paramètres) {
 - ...
 - *déclarations des variables locales*
 - }
- }

Références et objets

- Un **objet** doit être considéré comme accessible par **référence**. L'**identificateur** d'un objet désigne une **référence** à l'objet et non l'objet lui-même.
- Ce sont ces **références** qui sont affectées ou passées comme arguments lors des appels de méthodes.

- *Exemple:*

```
Fraction f1, f2;
```

```
f1 = new Fraction;
```

```
f2 = f1;
```

f1 et *f2* sont des variables qui se réfèrent au **même objet**.

Cet objet existe en un **seul exemplaire** mais avec **deux références**.

Les membres de classe

- ✓ **Les attributs ou méthodes sont accessibles:**
 - soit via une instance de la classe: **membres d'instance**
 - soit via la classe elle-même: **membres de classe** ou **membres statiques**.

- ✓ **Les membres de classe :**
 - sont introduits par le mot **static**
 - existent en un seul exemplaire
 - sont partagés par toutes les instances de la classe.

- ✓ **Ils ne peuvent être invoqués que sur la classe.**

Attributs de classe

- Ils sont initialisés au moment du chargement de la classe.
- Ils sont modifiables par tous les objets de la classe.
- Toute modification est répercutée au niveau des autres objets.
- Ils permettent d'éviter la duplication de certains types d'attributs (comme les constantes) dans chaque instance.

Exemple

L'attribut **out** est un attribut de la classe **java.lang.System**
System.out.println(" Salut! ");

Les méthodes de classe

- Elles ne peuvent pas modifier des attributs d'instance.
- Lorsqu'une méthode ne manipule que des attributs (ou méthodes) de classe, elle doit être déclarée **statique**.

Exemples

- **exit()** est une méthode statique de la classe **java.lang.System**

```
System.exit();
```

- **sqrt()** est une méthode statique de la classe **java.lang.Math**

```
System.out.println(Math.sqrt(100));
```

Les tableaux (1)

✓ Les tableaux se comportent un peu comme des objets.

✓ Deux syntaxes pour la déclaration:

TypeDesElements[] NomDuTableau;

TypeDesElements NomDuTableau[];

✓ Exemple :

`int[ ] tab_entiers; ou int tab_entiers[ ];`
`String[ ] tab_chaines;`

✓ Le type des éléments du tableau peut être une classe existante.

Les tableaux (2)

- Instanciación

*TypeDesElements[] NomDuTableau =
new TypeDesElements[TailleDuTableau];*

Exemple

- *int[] Tab_entiers = new int[15];*
- *String[] Chaines = new String[3];*

- Matrices

- *TypeDesElements[][] NomDuTableau;*
- *TypeDesElements NomDuTableau[][];*

Exemple

- *int [][] matrice*
- *ou int matrice [][];*

Les tableaux (3)

Exemple de saisie d'un tableau:

```
public static void main(String[] args) {  
  
    int[] t = new int[5];  
  
    BufferedReader e = new BufferedReader  
        (new InputStreamReader(System.in));  
  
    for (int i=0; i<5; i++)  
        t[i] = Integer.parseInt(e.readLine());  
}
```



PACKAGES

Les packages : définition et utilité

Définition

- Un package est un ensemble de classes et d'autres packages regroupés sous un nom.
- C'est l'adaptation du concept de librairie ou de bibliothèque.

Utilité

- Servent à structurer l'ensemble des classes et interfaces.
- Augmentent la lisibilité des applications en structurant l'ensemble des classes selon une arborescence.
- Facilitent la recherche de l'emplacement physique des classes
- Empêchent la confusion entre des classes de même nom
- etc.

Les packages : généralités

- Lors de la définition d'un fichier source Java, on peut préciser son appartenance à un package en utilisant le mot-clé **package**:

- **package** mon_package;

- Toutes les classes définies dans ce fichier font ainsi partie du package **mon_package**.

exemple: `package formesGeo;`

- Si la classe Carre se trouve dans un package formesGeo,
 - son nom complet est formesGeo.Carre;
 - le fichier Carre.class doit être placé dans un répertoire nommé formesGeo

Les packages : importation

- Pour pouvoir utiliser une classe d'un package dans un autre fichier, il faut l'importer:
 - `import nom_du_package.nom_classe;`
- On peut importer toutes les classes d'un package en même temps:
 - `import nom_du_package.*;`
`import java.io.*;`
`import formesGeo.Carre;`
- Le package `java.lang` qui gère les types de données et les éléments de base du langage est automatiquement importé.

Les packages : compilation

- Pour compiler une classe contenant l'instruction suivante, il faut indiquer le chemin au compilateur.
 - **import** monPackage.MaClasse;
- *soit* : **Option `-classpath` de javac**
`javac -classpath chemin_du_fichier MaClasse.class MaClasse`
- *soit* : **Variable d'environnement CLASSPATH**
doit contenir le chemin d'accès au répertoire racine du package.

Les packages : exécution

Pour exécuter la commande java
monPackage.MaClasse :

- *soit* : Se placer dans le répertoire contenant le répertoire monPackage
- *soit* : **Utiliser l'option `-classpath` de la commande java**
- *soit* : **Variable d'environnement `CLASSPATH`**
doit contenir le chemin d'accès au répertoire monPackage.

Les packages prédéfinis (1)

- Les API Java sont organisées en deux parties :
 - **Java Core API** : API de base implantée dans tout interpréteur Java. Ces classes appartiennent au package “java”.
 - **Java Standard Extension API** : extensions normalisées par Sun au travers d’interfaces et implantées par les éditeurs.

Ces classes appartiennent au package “javax”.

- Documentation sur les toutes classes Java disponible sur le site officiel de Java : <http://java.sun.com/>

Les packages prédéfinis (2)

Les classes sont regroupées par thèmes dans des packages :

- **java.lang** : types de données et éléments de base du langage (classes **Object**, String, Boolean, Integer, Float, Math, System (fonctions système), Runtime (mémoire, processus), etc.)
- **java.util**: structures de données et utilitaires (Dates, Collections, etc.)
- **java.io**: bibliothèque des entrées / sorties, fichiers.
- **java.net**: bibliothèque réseau
- **java.awt**: librairie graphique
- **java.applet**: librairie des applets.
- **java.security**: sécurité (contrôle d'accès, etc.)
- **java.rmi**: applications distribuées.
- **java.sql**: accès aux BD (JDBC)



EXCEPTIONS

Les exceptions (1)

- Une **exception** est un **signal** qui indique qu'un évènement exceptionnel comme une erreur est survenue au cours de l'exécution d'un programme :
 - **FileNotFoundException** survient lorsqu'on tente d'ouvrir un fichier existant
 - **IOException** survient lorsqu'on fait une lecture incorrecte, lorsqu'un fichier ne peut être fermé, etc.
- Java permet de gérer les erreurs par exception, i.e. de prévoir dans son application des blocs de code où sera traitée systématiquement, telle ou telle condition d'erreur.
- Une exception est un objet de la classe **java.lang.Exception**.
- Pour les gérer, on utilise l'**instruction try**.

Les exceptions (2)

```
try {  
    // bloc d'instructions à protéger  
}  
  
catch ( Exception e) {  
    // traitement de l'exception  
}  
  
finally {  
    // à exécuter quelque soit la façon dont on sort du try  
}
```

Les exceptions (3)

- Un bloc d'instructions **try** permet de regrouper une ou plusieurs instructions de programme (appel de méthode, instructions de contrôle, etc.) où des exceptions peuvent se produire et être traitées.
- Un bloc **try** est suivi d'instructions ***catch*** et ***finally***.
- **catch** spécifie le traitement associé aux exceptions, i.e ce que l'on fait si une exception se produit (afficher un message d'erreur par exemple).
- **finally** est exécuté inconditionnellement, quelque soit la façon dont on sort du bloc **try** (pour fermer par exemple un fichier ouvert dans le bloc **try**).

Les exceptions (4)

```
import java.io.*;

public class lectureFichier {
    public static void main(String[] args) {
        try{
            FileReader f=new FileReader("fic.txt");
            System.out.println("Le fichier existe !");
            f.close();
            System.out.println("Le fichier a ete
ferme !");
        }
        catch(FileNotFoundException fe){
            System.out.println("Fichier
inexistant !");
        }
        catch(IOException ie){
            System.out.println("Erreur de
fermeture !");
        }
    }
}
```

Les exceptions (5)

Chaque **méthode susceptible de générer une exception** qui lui est propre doit la déclarer dans son en-tête. On utilise l'instruction ***throws*** pour cela :

```
public class Pile {  
    final int taille_max = 100 ;  
    private int sommet ;  
    private int[ ] elements ;  
  
    public void empiler (int x) throws ErreurPilePleine {  
        if (sommet == taille_max)  
            throw new ErreurPilePleine  
        else    elements[++sommet] = x ;  
    }  
}  
  
class ErreurPilePleine extends java.lang.Exception {  
    ErreurPilePleine(){ System.out.println("Pile Pleine");}  
}
```

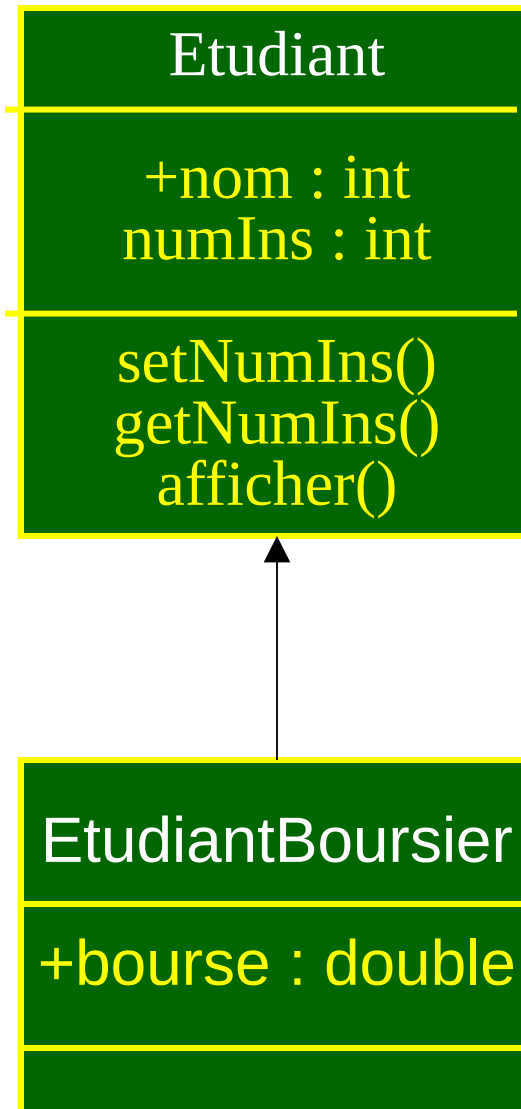


HERITAGE

L'héritage

- L'héritage permet de définir une nouvelle classe à partir d'une (ou plusieurs) classe(s) existante(s).
- Les classes sont souvent organisées en hiérarchies; toutes les classes Java héritent de la classe **java.lang.Object**.
- Java n'autorise que l'héritage simple.
- L'héritage multiple est “remplacé” par la notion d'**interface**.

L'héritage simple : exemple



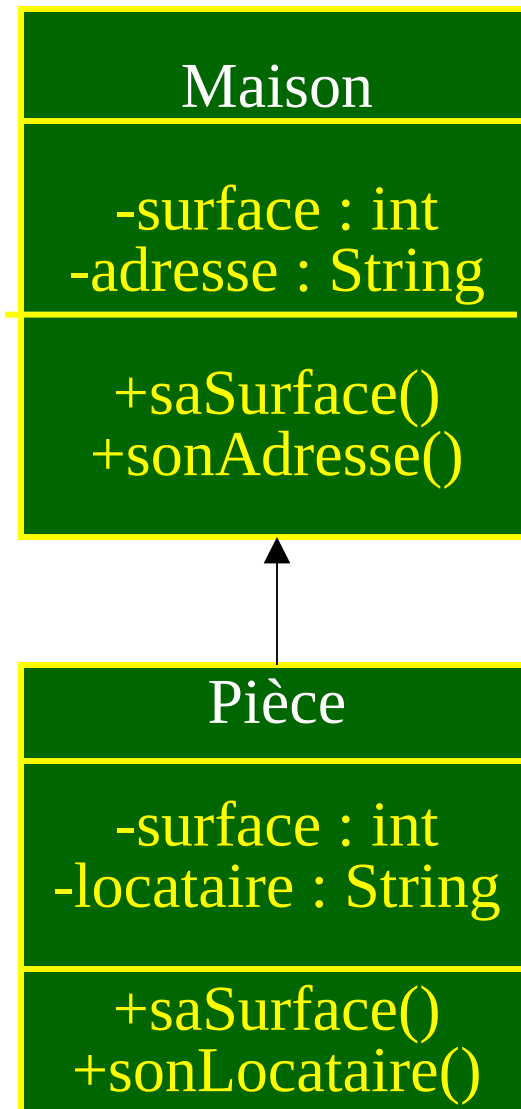
L'héritage simple : exemple

```
public class EtudiantBoursier extends Etudiant {  
  
    public double bourse;  
  
    public EtudiantBoursier(String nom, double bourse) {  
        super(nom); //appel au constructeur Etudiant(String)  
        this.bourse = bourse;  
    }  
    public EtudiantBoursier(String nom) {  
        this(nom, 0); //appel au constructeur précédant  
    }  
  
    public void afficher() { // redefinition  
        System.out.println("etudiant :" + this.nom);  
        System.out.println("bourse :" + this.bourse);  
    }  
}
```

La pseudo-variable super

- En faisant hériter une classe d'une autre, on peut définir une méthode dont l'identificateur est le même que celui sa classe mère.
- masquage de la méthode de la classe mère.
- La pseudo-variable **super** permet d'accéder à la méthode masquée de la classe mère.

La pseudo-variable super : exemple



Les classes abstraites

- Elles permettent de définir l'interface des méthodes.
- Elles contiennent au moins une **méthode abstraite**.
- Une méthode abstraite est précédée du mot-clé **abstract** et ne possède pas de corps.
- Elles ne peuvent pas êtreinstanciées.
- Elles doivent être dérivées en sous-classes fournissant une implémentation à toutes les méthodes abstraites.
- Elles sont définies par l'introduction du mot-clé **abstract**.

```
abstract class FormeGeometrique {  
    abstract double perimetre();  
    abstract double surface();  
}
```

Les interfaces (1)

- Ce sont des classes abstraites dont l'instanciation serait sans intérêt. Elles ne peuvent donc pas être instanciées.
- Elles peuvent hériter d'autres interfaces (héritage simple).
- Elles ne peuvent contenir que des attributs statiques constants (*static final*) et des méthodes publiques non implémentées.
- Elles doivent être **implémentées** par une ou plusieurs classes.
- Une classe peut implémenter une ou plusieurs interfaces.
- Lorsqu'une classe implémente une interface, elle doit implémenter toutes les méthodes définies dans l'interface.

Les interfaces (2)

```
public interface FormeGeometrique {  
    public double perimetre();  
    public double surface();  
}
```

```
public class Rectangle implements FormeGeometrique {  
  
    public double larg, long;  
    public Rectangle(double long, double larg) {  
        this.long=long; this.larg=larg;  
    }  
    public double perimetre() {return 2*(larg+long);}  
    public double surface() {return larg*long;}  
}
```

Les interfaces (3)

```
public class Carre implements FormeGeometrique {  
    public double cote;  
    public Carre(double cote) {this.cote = cote;}  
    public double perimetre() {return 4*cote;}  
    public double surface() {return cote*cote;}  
}  
  
public class Cercle implements FormeGeometrique {  
    public final static double PI = 3.1416;  
    public double rayon;  
    public Cercle (double rayon) {this.rayon = rayon;}  
    public double perimetre() {return 2*PI*rayon;}  
    public double surface() {return PI*rayon*rayon;}  
    public void afficher() {  
        System.out.println("cercle de rayon"+rayon); }  
}
```

Les interfaces (4)

Exemple d'utilisation :

```
public class Figures {  
    public static void main (String[] args) {  
        FormeGeometrique[] formes;  
        formes = new FormeGeometrique[3];  
        formes[0] = new Cercle(10);  
        formes[1] = new Carre(4);  
        formes[2] = new Rectangle(5,8);  
        for (int i=0;i<formes.length;i++){  
            System.out.print("surface de la  
forme"+i+" : ");  
            System.out.println(formes[i].surface());  
        }  
    }  
}
```

La classe Object

- Toutes les classes héritent de la classe Object qui contient certaines propriétés et méthodes intéressantes permettant, par exemple :
 - Connaître la classe d'un objet : getClass()
 - Comparer des objets : equals()
 - Afficher des objets : toString()

- *Exemple:*

```
public class Fraction {  
    public int num, den ;  
    public Fraction (int n, int d) { this.num=n; this.den=d;}  
    public String toString() {  
        return this.num + "/" + this.den;  
    }  
}
```

Fin du Chapitre 2.